专业：计算机科学与技术
姓名：方彦祺
学号：3220102829
日期：2025.1.1
地点：无

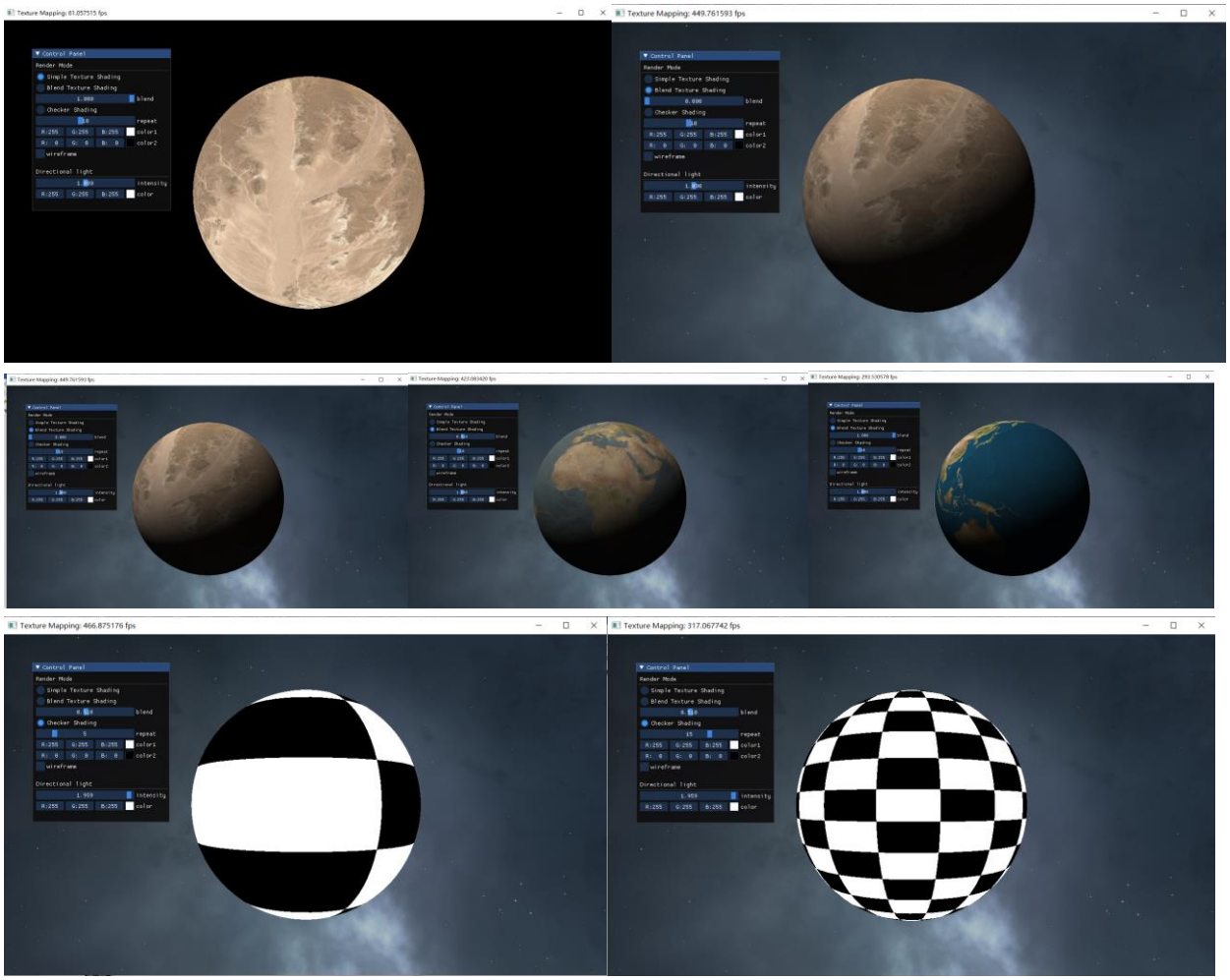
浙江大学实验报告

课程名称：计算机图形学 指导老师：童若锋 成绩：
实验名称：OpenGL 纹理映射 实验类型：基础实验 同组学生姓名：无

装 订 线

一、实验目的和要求

掌握纹理映射的基本原理，并实现纹理混合、程序纹理与天空盒的绘制。



二、实验内容和原理

本次实验内容包括了解纹理的基本概念，学习如何在 OpenGL 中加载和应用纹理，以及如何实现纹理混合和天空盒的绘制。原理涉及纹理坐标的设置、纹理采样、纹理过滤和环绕方式等。纹理映射利用纹理坐标来指定图像上的颜色如何映射到模型的几何表面上。纹理坐标是一个二维值（在 3D 中可以扩展到三个维度），用于定义纹理图像上的特定点如何对应到 3D 模型的顶点上。纹理采样是纹理映射过程中的一个步骤，它涉及根据纹理坐标从纹理图像中获取颜色值。纹理过滤决定了当纹理像素映射到屏幕像素时，如何对纹理进行缩放和放大。立方体贴图是一种特殊的纹理，它由六个 2D 纹理组成，每个纹理代表立方体的一个面。立方体贴图常用于实现天空盒和环境映射效果。立方体贴图的采样是基于方向向量的，这意味着可以根据观察方向来获取纹理颜色。

三、主要仪器设备

Visual Studio 2022

四、操作方法和实验步骤

```
"void main() {\n"
"    vec4 color1 = texture(mapKds[0], fTexCoord) * vec4(material.kds[0], 1.0f);\n"
"    vec4 color2 = texture(mapKds[1], fTexCoord) * vec4(material.kds[1], 1.0f);\n"

"    vec4 blendedColor = mix(color1, color2, material.blend);\n"

"    vec3 normal = normalize(fNormal);\n"
"    vec3 lightDir = normalize(-light.direction);\n"
"    float lambert = max(dot(normal, lightDir), 0.0);\n"

"    vec3 lighting = lambert * light.color * light.intensity;\n"

"    color = vec4(blendedColor.rgb * lighting, blendedColor.a);\n"
"}\n";
```

这段代码中：

- vec4 color1 = texture(mapKds[0], fTexCoord) * vec4(material.kds[0], 1.0f);：采样第一个纹理，并乘以材料的颜色。
- vec4 color2 = texture(mapKds[1], fTexCoord) * vec4(material.kds[1], 1.0f);：采样第二个纹理，并乘以材料的颜色。
- vec4 blendedColor = mix(color1, color2, material.blend);：根据混合系数 material.blend 混合两个颜色。
- vec3 normal = normalize(fNormal);：规范化法线向量。
- vec3 lightDir = normalize(-light.direction);：规范化光源方向向量。
- float lambert = max(dot(normal, lightDir), 0.0);：计算 Lambert 光照模型的漫反射项。
- vec3 lighting = lambert * light.color * light.intensity;：计算最终的光照颜色。
- color = vec4(blendedColor.rgb * lighting, blendedColor.a);：将混合颜色与光照颜色相乘，并保留混合颜色的 alpha 值。

```
"void main() {\n"

"    int x = int(fTexCoord.x * material.repeat);\n"
"    int y = int(fTexCoord.y * material.repeat);\n"

"    bool isWhite = (x + y) % 2 == 0;\n"
"    vec3 checkerColor = isWhite ? material.colors[0] : material.colors[1];\n"

"    color = vec4(checkerColor, 1.0f);\n"
"}\n";
```

这段代码中：

计算纹理坐标 fTexCoord 的 x 分量乘以材料的重复因子 material.repeat，并将结果转换为整数。这用于确定棋盘格在 x 方向上的单元格编号；同样计算纹理坐标的 y 分量乘以重复因子，并将结果转换为整数，用于确定棋盘格在 y 方向上的单元格编号。然后通过将 x 和 y 坐标相加并检查结果是否为偶数来确定当前单元格的颜色。使用三元运算符根据 isWhite 的值选择颜色。如果 isWhite 为真，则 checkerColor 被设置为 material.colors[0]；如果为假，则设置为 material.colors[1]。最后，这行代码将计算出的 checkerColor 与 1.0f（不透明度）结合，创建一个 RGBA 颜色向量，并将其赋值给 color 变量。color 是片段着色器的输出，它决定了最终渲染到屏幕上的像素颜色。

```
_blendMaterial->mapKds[0]->bind(0);
_blendMaterial->mapKds[1]->bind(1);
_blendShader->setUniformInt("mapKds[0]", 0);
_blendShader->setUniformInt("mapKds[1]", 1);
```

调用了第一个纹理对象 mapKds[0]的 bind 方法，并传递了纹理单元编号 0 作为参数。这将纹理绑定到 OpenGL 的纹理单元 0 上，使得该纹理可以在着色器中通过纹理单元 0 进行采样。将第二个纹理对象 mapKds[1]绑定到纹理单元 1 上。这样，两个纹理就可以在着色器中分别通过纹理单元 0 和 1 进行采样。使用着色器程序_blendShader 的 setUniformInt 方法设置 uniform 变量 mapKds[0]的值。这里，我们将纹理单元 0 的编号传递给着色器中的 mapKds[0] uniform 变量，这样着色器就知道 mapKds[0]对应的是纹理单元 0。设置着色器中的 mapKds[1] uniform 变量，将其与纹理单元 1 关联。这样，着色器在进行纹理采样时，会从纹理单元 1 中获取 mapKds[1]对应的纹理。

```
void SkyBox::draw(const glm::mat4& projection, const glm::mat4& view) {
    // TODO:: draw skybox
    // write your code here
    // -----------------------------------------------
    // ...
    // -----------------------------------------------
    glDepthFunc(GL_LEQUAL);

    _shader->use();

    glm::mat4 viewNoTranslation = glm::mat4(glm::mat3(view));
    _shader->setUniformMat4("projection", projection);
    _shader->setUniformMat4("view", viewNoTranslation);

    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_CUBE_MAP, _texture->getHandle());
    _shader->setUniformInt("cubemap", 0);

    glBindVertexArray(_vao);
    glDrawArrays(GL_TRIANGLES, 0, 36);
    glBindVertexArray(0);

    _shader->unuse();

    glDepthFunc(GL_LESS);
```

这段代码完成了禁用深度缓冲区写入；使用天空盒着色器；设置投影和视图矩阵；绑定天空盒纹理；

绘制立方体；恢复深度缓冲区写入。

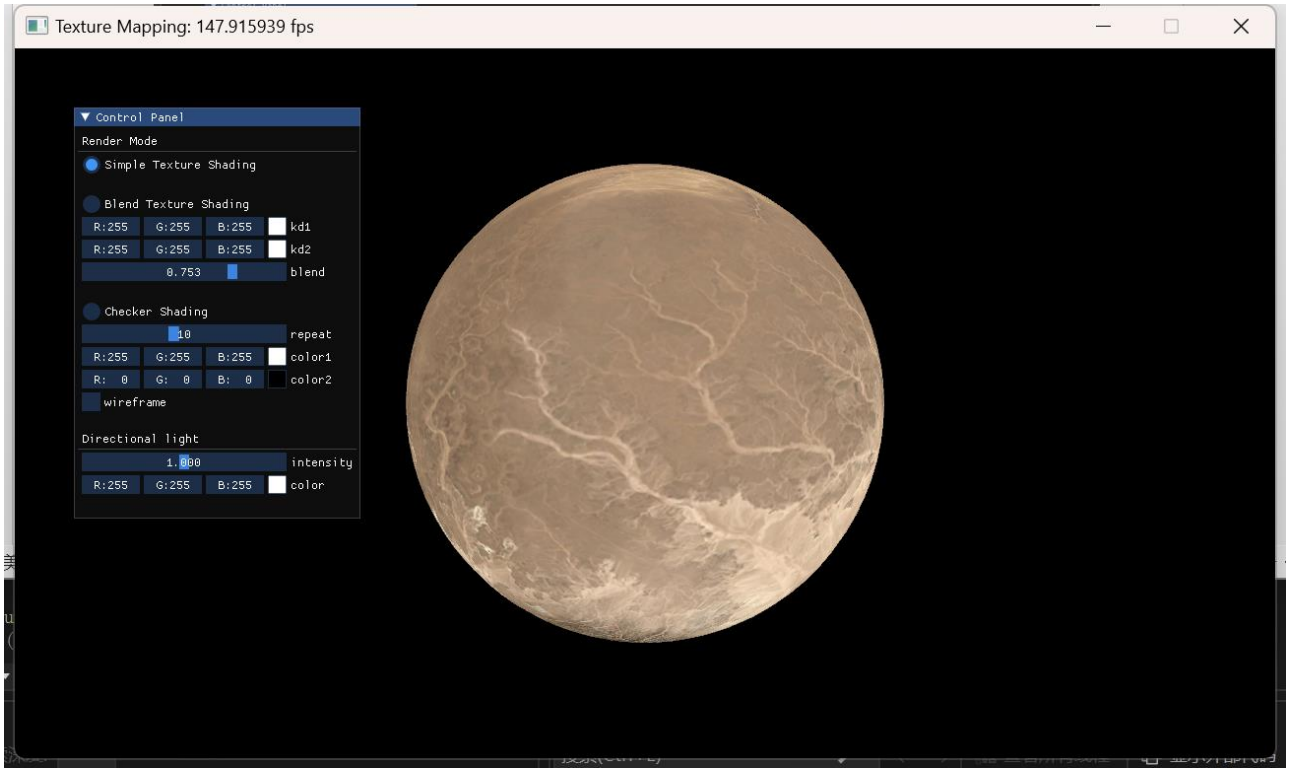
```
unsigned int textureID;
glGenTextures(1, &textureID);
glBindTexture(GL_TEXTURE_CUBE_MAP, textureID);

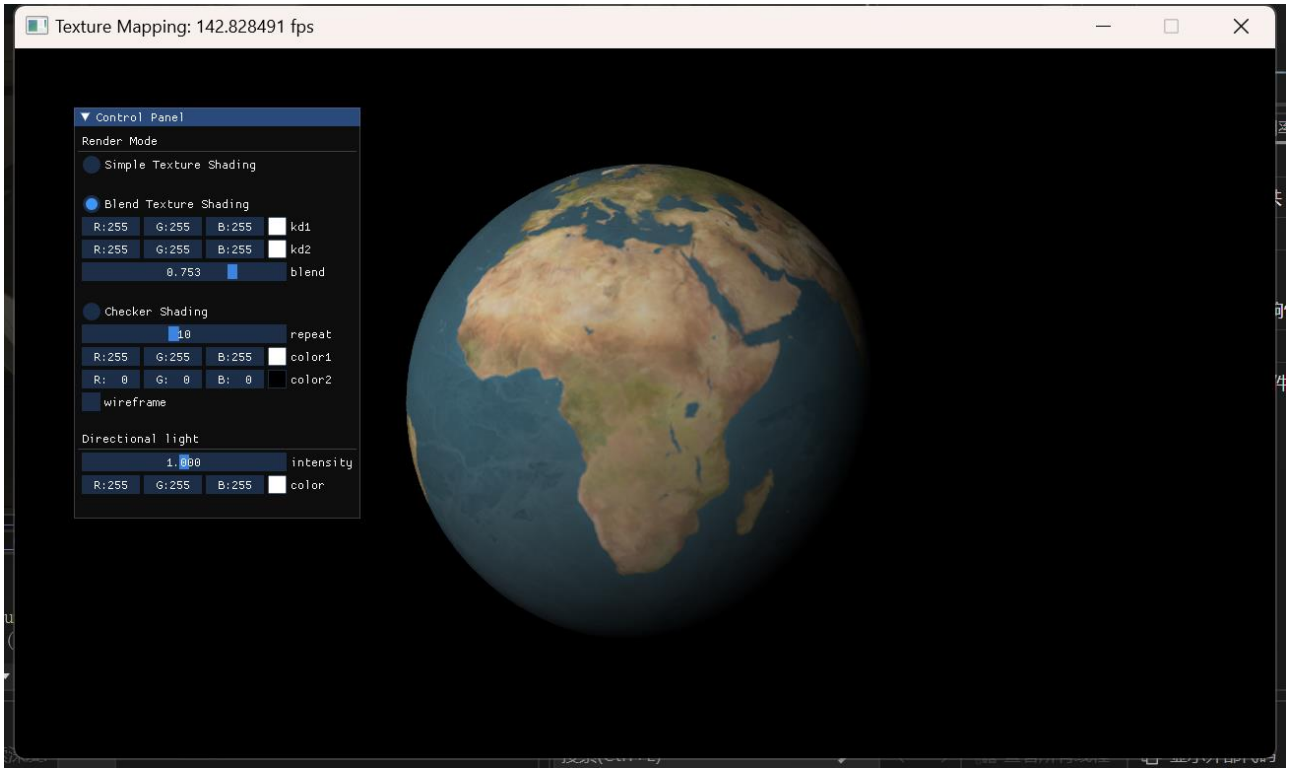
int width, height, nrChannels;
for (unsigned int i = 0; i < filepaths.size(); i++)
{
    unsigned char* data = stbi_load(filepaths[i].c_str(), &width, &height, &nrChannels, 0);
    if (data)
    {
        glTexImage2D(GL_TEXTURE_CUBE_MAP_POSITIVE_X + i,
            0, GL_RGB, width, height, 0, GL_RGB, GL_UNSIGNED_BYTE, data
        );
        stbi_image_free(data);
    }
    else
    {
        std::cout << "Cubemap texture failed to load at path: " << filepaths[i] << std::endl;
        stbi_image_free(data);
    }
}
glTexParameteri(GL_TEXTURE_CUBE_MAP, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_CUBE_MAP, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_CUBE_MAP, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
glTexParameteri(GL_TEXTURE_CUBE_MAP, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
glTexParameteri(GL_TEXTURE_CUBE_MAP, GL_TEXTURE_WRAP_R, GL_CLAMP_TO_EDGE);
```

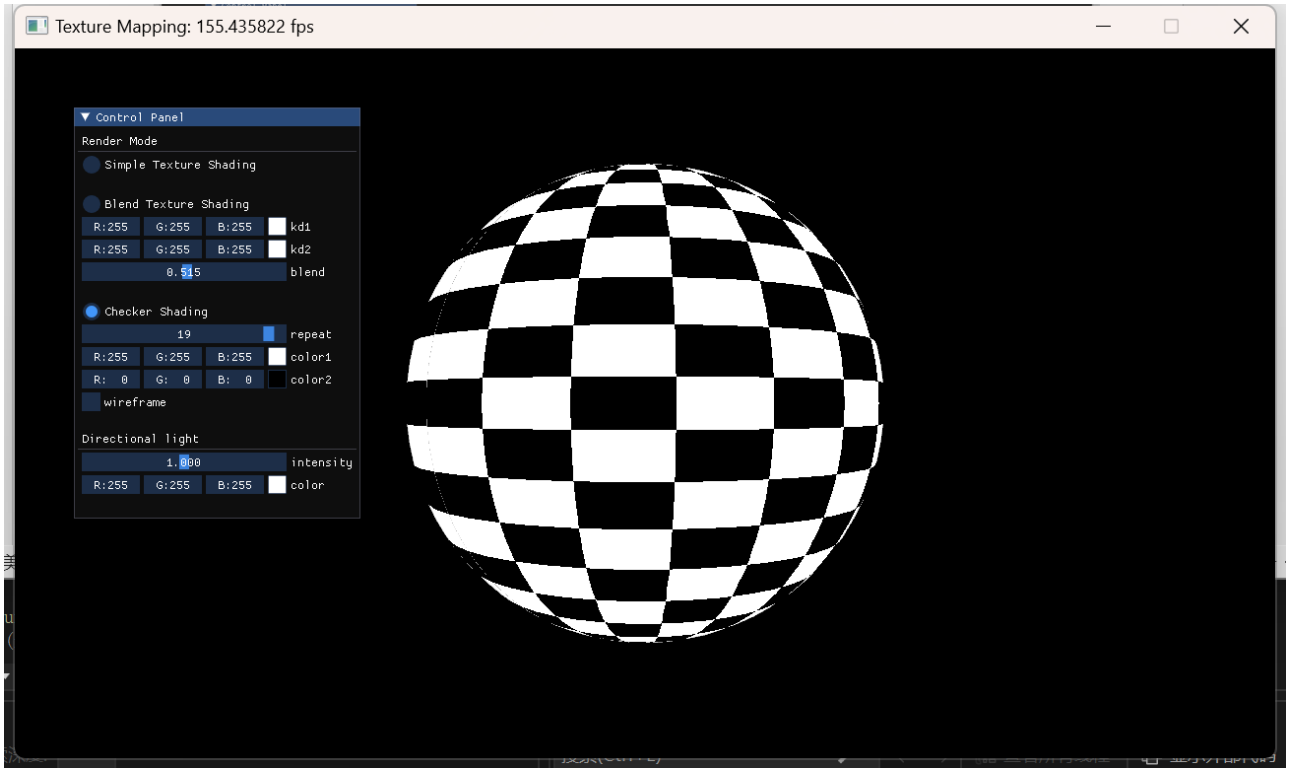
这段代码用来在 OpenGL 中加载并创建立方体贴图（Cube Map）。首先，定义一个无符号整数 textureID 来存储生成的纹理 ID。使用 glGenTextures 函数生成一个纹理对象，并将其 ID 存储在 textureID 中。使用 glBindTexture 函数将生成的纹理对象绑定到立方体贴图目标（GL_TEXTURE_CUBE_MAP）。定义三个整数变量 width、height 和 nrChannels 来存储纹理图像的宽度、高度和颜色通道数。遍历包含立方体贴图六个面纹理文件路径的 filepaths 向量。使用 stbi_load 函数加载每个面的纹理图像，并将图像数据存储在 data 指针中。如果 data 不为空，说明纹理加载成功。使用 glTexImage2D 函数将加载的纹理图像数据指定给立方体贴图的相应面（GL_TEXTURE_CUBE_MAP_POSITIVE_X 到 GL_TEXTURE_CUBE_MAP_NEGATIVE_Z）。
纹理参数设置为 RGB 格式，不进行 mipmap 处理（mipmap 级别为 0）。
调用 stbi_image_free 释放加载的图像数据。
如果 data 为空，说明纹理加载失败，输出错误信息到控制台，并尝试释放可能已经分配的图像数据。
设置立方体贴图的纹理过滤参数，这里设置为线性过滤（GL_LINEAR），用于缩小和放大纹理时的纹理采样。
设置立方体贴图的纹理环绕参数，这里设置为边缘环绕（GL_CLAMP_TO_EDGE），这意味着纹理坐标超出[0, 1]范围时，将使用最接近的边缘像素颜色进行渲染。

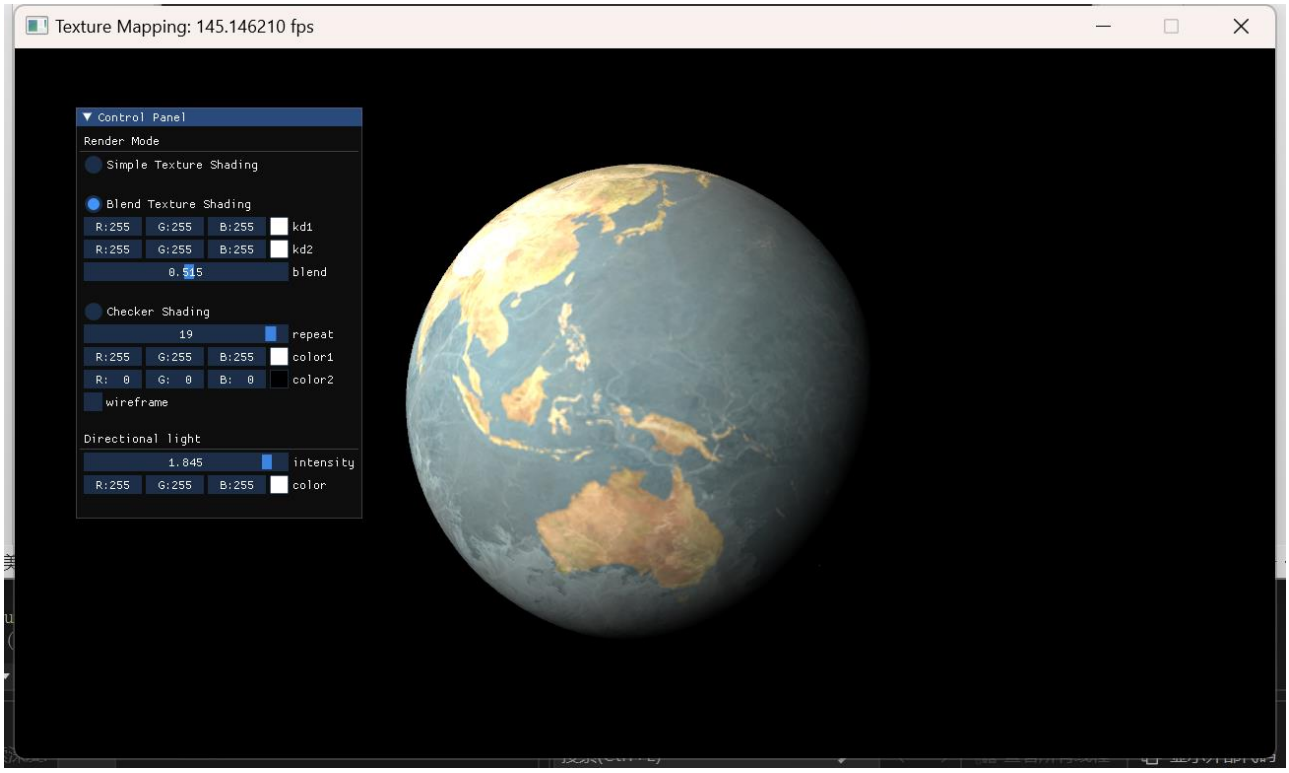
五、实验结果与分析

见生成的可执行文件，这里给出一些操作的截图：


Texture Mapping: 147.915939 fps
Control Panel
Render Mode
Simple Texture Shading
Blend Texture Shading
R:255 G:255 B:255 kd1
R:255 G:255 B:255 kd2
0.753 blend
Checker Shading
10 repeat
R:255 G:255 B:255 color1
R: 0 G: 0 B: 0 color2
wireframe
Directional light
1.000 intensity
R:255 G:255 B:255 color


Texture Mapping: 142.828491 fps
Control Panel
Render Mode
Simple Texture Shading
Blend Texture Shading
R:255 G:255 B:255 kd1
R:255 G:255 B:255 kd2
0.753 blend
Checker Shading
10 repeat
R:255 G:255 B:255 color1
R: 0 G: 0 B: 0 color2
wireframe
Directional light
1.000 intensity
R:255 G:255 B:255 color





六、讨论、心得

本次实验需要填写的部分较少，原理也较为简单，能够比较顺利地完成实验。

七、参考链接

[1] 纹理 - LearnOpenGL CN (learnopengl-cn.github.io)

[2] 立方体贴图 - LearnOpenGL CN (learnopengl-cn.github.io)